



Software Requirements Specification

Ponnangai POS

Date: May 10, 2026

Version: 1.0

ABSTRACT

Traditional retail environments often rely on manual billing and disjointed inventory tracking, leading to inefficiencies and data loss. This project presents the design and implementation of Ponnangai POS, a centralized, mobile-first Point of Sale and retail management system. Built using a Python (FastAPI) backend and an SQLite database, the system serves a highly responsive, flat-UI frontend tailored for rapid mobile data entry. The architecture implements strict Role-Based Access Control (RBAC) to separate shopkeeper terminal operations from administrative analytics and inventory management. A key innovation of this system is its hardware integration, specifically optimizing web-based browser print outputs to interface seamlessly with 58mm Bluetooth thermal POS printers. By digitizing the billing workflow and providing real-time, mathematically isolated inventory sync across multiple users, Ponnangai POS modernizes daily shop operations and provides stakeholders with actionable, data-driven insights.

1. Introduction

1.1 Purpose

The purpose of this document is to define the software requirements for Ponnangai POS, a centralized, mobile-first Point of Sale (POS) and retail billing system designed for "Ponnangai Enterprises". This document outlines the system architecture, business rules, edge cases, and hardware requirements.

1.2 Scope

Ponnangai POS is a web-based retail management application designed to handle multi-tenant retail operations. It provides an intuitive, touch-friendly interface for shopkeepers to process sales and a powerful administrative dashboard for owners to track revenue, manage inventory, and monitor staff performance.

1.3 Technology Stack & Hosting

- **Backend:** Python, FastAPI (Hosted on a cloud provider such as Render or PythonAnywhere, serving the backend API and database).
- **Database:** SQLite (managed via SQLAlchemy ORM).
- **Frontend:** Vanilla HTML, CSS, JavaScript (Mobile-optimized frontend served dynamically by FastAPI).
- **Domain:** Linked via Namecheap DNS records to the cloud hosting provider.

2. Overall Description

2.1 User Classes and Characteristics

The system implements strict Role-Based Access Control (RBAC) with four distinct user types:

1. **Shopkeeper:** Operates the POS terminal. Can view their assigned inventory, generate bills, print receipts, and manage their recent bill history (revert/cancel).
2. **Manager:** Oversees daily operations. Can add new Shopkeeper accounts, view shop analytics, and manage inventory (bulk upload and individual stock updates).
3. **Admin:** Full system control. Can edit/delete all user accounts, reset passwords, view all analytics, and execute dangerous operations (System DB Reset).
4. **Owner:** Executive overview. Has read/write access to analytics, inventory, and users, but lacks dangerous database override permissions.

2.2 Hardware & Environment

- **Server:** Python 3.9+ environment.
- **Client (POS):** Any mobile device or tablet with an active internet connection.
- **Printer Integration:** Optimized specifically for the Niyama BT-58 (or similar 58mm Bluetooth/USB thermal printers).

3. System Features & Business Rules

3.1 Point of Sale (POS) Interface

- **Flat UI Design:** Inspired by Blinkit's flat, clean interface for rapid tapping. It currently operates as a unified product grid for speed, with the future scope of categorical division if inventory scales significantly.
- **Dynamic Cart Management:** Tap to add, adjust quantities, apply manual monetary discounts, and select payment modes (Cash/UPI).

3.2 Printed Bill Format (58mm Receipt)

The receipt strictly adheres to the 58mm layout and prints the following:

- **Header:** "Ponnangai Enterprises" alongside the subtitle "Housekeeping Products". (Future Scope: Configurable GSTIN and Physical Address placeholders).
- **Metadata:** Bill Number, accurate Timestamp, Payment Mode (Cash/UPI).
- **GST Compliance:** The exact Cashier Name is permanently printed and logged on the receipt at the time of sale. This satisfies GST sales reporting requirements even if that cashier's account is deleted from the system later.
- **Financials:** Itemized breakdown, Subtotal, Applied Discounts, and Final Total.
- **Footer:** A greeting message: "Thank you for shopping! Please visit again!"

3.3 Return & Revert Workflow (Edge Cases)

To handle customer returns and cancellations, a Bill History Sidebar is integrated directly into the POS screen:

- **"Cancel Bill"**: Used when a customer completely returns their purchase. The system instantly voids the bill, deletes the record, and restores the specific product quantities back to that shop's inventory.
- **"Revert & Edit"**: Used for modifications. Voids the historical transaction, restores the inventory, and instantly repopulates the active cart with the returned items so the shopkeeper can modify and reprint the bill seamlessly.

3.4 Multi-Shop Inventory Management & Alerts

- **Shop-Wise Isolation**: Each shopkeeper operates a mathematically isolated inventory.
- **Bulk CSV Operations**: Managers can download template files, adjust quantities, and upload them to update hundreds of items instantly.
- **Low Stock Warnings**: The admin dashboard globally monitors all shops and triggers a red alert when any product falls below 10 units at any specific location, ensuring management can proactively restock.

3.5 Centralized Admin Dashboard & Analytics

- **Global Metrics:** Tracks overall total revenue and total bills generated across the entire enterprise in real-time.
- **Shop-Specific Analytics:** Clickable shop performance reports. Selecting a shop opens a detailed modal showing exact itemized sales and revenue breakdowns for that specific location.
- **(Future Scope:** Implementing Date-filters (Daily/Weekly/Monthly) and CSV Export buttons for the Analytics views to assist with accounting).

4. Non-Functional Requirements & Edge Cases

4.1 Internet Drop & Connectivity

- As a centralized web application, an active internet connection is required to process bills and sync inventory. If the mobile internet drops while creating a bill, the browser will hold the cart state locally. Once the connection is restored, the shopkeeper can tap "Confirm & Print" to push the transaction to the server.

4.2 Database Integrity

- **User Deletion:** If an Admin deletes a Shopkeeper account, the shopkeeper's unsold inventory is wiped. However, their historical sales (Bills) are retained in the database to ensure global revenue analytics do not drop, and the cashier_name remains tied to those bills for tax audits.

4.3 Performance

- The POS screen utilizes asynchronous API calls (fetch) to ensure the UI never blocks while interacting with the database, maintaining a highly responsive, app-like feel.